

卷	副标题（连载）	你得到什么
—	从「更会写」到「敢合并」	从「更会写」到「敢合并」· 15 分钟导读
卷一	怎样才算「做完」	动机、双轨总览、最小起步
卷二	技术图谱	子图、读法对照、图谱 CI
卷三	Harness 与 SDD	实践 SDD 的 Harness 协作流程（任务单、签收、阶段流）
卷四	闭环交付与经验沉淀	专题流水线、跨轮回顾摘要
卷五	存量怎么落地	案例机制、FAQ、阶段 0~3、诚实边界

目录

节	标题
—	摘要
11	任务单最少写什么
12	审查与签收
13	SDD 阶段流
14	半自动协作
15	存量 / 无 CI 降级
16	结语

摘要

若你还没读过 **卷一** 与 **卷二**，建议先看——卷一讲 **图谱 + 协作流程** 如何叠放；卷二讲 Agent **先看地图再动手**。

本卷只展开过程这一轨：任务单最少写什么、为什么 **书面审查 / 签收** 不可省、**规格驱动（SDD）** 阶段如何从澄清走到合并、以及 **半自动协作** 的边界。

卷二解决「改哪里」；卷三解决「何时算做完、谁签字、凭什么合并」。**有地图没签收，照样不敢合并**。

本卷还回答一个行业难题：**用 AI 写代码，谁负责、如何证明尽到了注意义务、出问题从哪修**——答案不在更大的模型里，而在 **任务单 + 书面签收 + 合并前自动检查** 组成的 **可追责轨迹**（下文 §12.8）。

收尾后的经验沉淀在 **卷四**；**冷/温/热分层用语**在本卷首次引入；本卷聚焦 **温层里的签收纪律** 与日常默认操作（分层对照表见 §11.2.1）。

11. 任务单最少写什么：验收、非范围、失败路径

11.1 本节要回答什么

一轮需求 开工前，任务单里 最少 要有哪些内容，人和 Agent 才能对齐「什么叫做完」——而不是在聊天里各说各话。

11.2 Harness 是什么

Harness 协作流程 指：用 可检索的任务单 + 书面审查 + 合并前自动检查，把「聊过了」变成「交差了」。关键是 字段与签收纪律，不是多买一个工具。

它与卷一的三支柱对应：

支柱	任务单里体现什么
告知	背景、范围、图谱入口、依赖说明
约束	非范围、失败路径、测试策略
验证	验收清单里的 合并前必绿 + 自检命令

11.2.1 图谱入口与冷/温/热分层（本卷用语 · 衔接卷二）

分层用语在本卷首次出现：卷一、卷二已分别讲 结构（技术图谱） 与 过程骨架，但正文 未使用「冷层 / 温层 / 热层」命名。下表是把前文 对照收拢 的简称。

层	是什么	与已读前卷的关系	本卷落点
冷层	不常变的 结构地图	对应卷二的技术图谱（卷二称「地图」「子图」「图谱入口」）	任务单里的 图谱入口
温层	协作轨迹：任务单 + 书面签收 + 回顾摘要	卷一过程轨 + 本卷主体	§11—§14
热层	运行时事件记忆（远期、非日常必做）	前卷未展开	§14.8 划界；完整愿景另有单独篇章详细介绍。

若你已读过卷二，可把那里的 技术图谱 理解为 冷层 的落点：回答 改哪里、从哪进、会影响谁。任务单里的 图谱入口 一行，就是让 Agent 开工前先 挂到地图上，而不是在全仓库里乱搜。

冷层只管结构，不管「上次为何定这个阈值」——那是 温层 里回顾摘要的事（§13.7、卷四）。二者别混：有地图没签收，仍不敢合并。

与卷五的关系：冷/温/热的 完整定义 与「不是 架构三层」纠偏，见 [卷五 §23.1](#)（以卷五为准；本节仅为本卷协作记忆分层简称，不改卷三已发单卷正文结构）。

11.3 与卷一的关系

卷一 §3 用 **意图 / 成果 / 验收** 描述一轮交付；卷一 §6 给了最小起步示例。卷三把三要素 **落到可执行字段**：

卷一要素	任务单里对应什么
意图	背景与目标 + 非范围
成果	范围清单 + (建议) 图谱入口 (卷二 §8.3)
验收	可勾选清单 + 测试策略 + 合并前自动检查

11.4 最少字段表

任务单开工前，至少建议写清下面几项（名称可按团队习惯调整）：

项目	是什么	举例
验收清单	可勾选的完成条件	「PR 上单测 workflow 全绿」；「错误验证码返回 401」
非范围	本轮 明确不做 的事	「不改短信登录」
失败路径	出错时系统 应如何表现	「库不可用 → 500 + 可重试提示」
测试策略	关键路径是否 先写失败测试	见 §11.5
图谱入口 (建议)	先读哪张主图/子图	「从登录子图进入」

失败路径 建议表格式，每行：**触发** → **行为 (含状态码)** → **可否重试** → **用户可见类型**；可选加 **可测场景编号** 列 (如 `auth-invalid-code`)，与单测或验收命令互链。缺这一节，Agent 容易只写 happy path。

11.5 测试策略：写清档位，而非口号式 TDD

档位	含义	适用
必须自动化	先 可失败 的测试，再改实现；自检附命令与通过证明	鉴权、对外契约、流式背压、核心回归
建议有测	鼓励补测；验收以命令 + 人工为主	一般功能
不适用	一行理由	纯文档、无行为变更

一人团队可对普通功能选「建议有测」，但 **鉴权、对外 API** 仍建议「必须自动化」。

Harness 整体是 SDD + 验证，不是要求每个 task 都走 strict red-green。多数 **纯文档、无行为变更** 的任务可选「不适用」——**整体安全网仍是合并前 CI 全量回归绿**；在鉴权、对外契约等 **关键点** 再用「必须自动化」把行为钉住。

「必须自动化」只应用于 **少数高风险路径**。日常更常见的是：**失败路径写清楚** → **实现时间 PR 补测** → **CI 回归绿** → **自检贴命令输出** (「建议有测」档)。纯函数、权限闸门、已知 bug 回归，才更值得

先写失败测试再改实现。

11.6 合并前必绿

验收清单建议 **固定一条** 与 CI 对齐，例如：

- 后端： `pytest` （或 PR 上等价 workflow）全绿
- 前端： `lint` + `test` + `build` 全绿

本地命令与 PR workflow 一致，避免「我机器过了、流水线没过」。

笔者在 **示例后端 API 项目** 中已落地：任务模板强制该条；**任务审核** 在开工前核对字段是否齐全（2026-05，收尾专题 PR #90）。你的仓库可从卷一 §6 模板加一行「合并前命令」起步。

11.7 扩展示例：验证码登录（延续卷一 §6）

要素	内容
非范围	不改短信登录；不调整会话 TTL
失败路径	验证码错误 → 401；过期 → 401 + 提示刷新
测试策略	必须自动化： <code>TestLoginWithCaptcha</code> 等
验收	单测绿 + CI 绿 + 审查签收

11.8 可复制任务单骨架（在卷一模板上增补）

```
## 任务单：<动词 + 范围>

### 验收清单
- [ ] <功能验收 1>
- [ ] PR 上单测 / 构建 workflow 全绿（本地等价：<你的命令>; 尚无 CI 时改用手动命令，输出贴进签收记录，见 §15）

### 非范围
- <明确不做事>

### 失败路径
| 可测场景编号（可选） | 触发 | 行为 | 可重试 | 用户可见 |
| --- | --- | --- | --- | --- |
| auth-invalid-code | <例：验证码错误> | 401 | 否 | 提示刷新 |

### 测试策略
- 必须自动化 / 建议有测 / 不适用（理由一行）

### 图谱入口（可选）
- 子图：<你的流程图文件名>
```

💡 常见疑问

- 和 Jira / 飞书工单有什么区别？ → 工单管 产品沟通；任务单管 工程交付。叠加使用，不互相替代。
- 一人团队也要写吗？ → 可以短，但 验收 + 非范围 + 失败路径 不可省——否则无法核对 Agent 是否越界。
- 是不是每个 task 都要 TDD？ → 否。写清测试策略档位 + 合并前 CI 回归 是底网；只有鉴权、对外契约等关键点才用「必须自动化」先失败测例再改实现。

12. 审查与签收：书面记录为什么不可省

12.1 本节要回答什么

为什么「聊过了」「LGTM」不等于 可以合并。

12.2 反模式

做法	问题
口头 Review	换人、换模型后 不可检索
聊天里「过了」	无对照 验收条 的证据
无记录合并	出事无法回答「谁批准、依据是什么」

12.3 两类书面记录

记录类型	何时	产出
任务审核	实现 开始前	范围与字段是否齐全；缺项 阻塞开工
审查签收	自检后、合并前	对照 diff、CI、任务单的 结束本轮 结论

二者 不替代 Code Review 看 diff，而是把结论 落盘。Agent 不能 代你终局签收；合并主干 须人拍板。

12.4 任务审核审什么（清单思路）

实现开始前，审核人宜逐项核对（可写进半页审查记录）：

- 验收清单是否 可观测（能勾选、能跑命令）
- 非范围 是否非空
- 失败路径 是否至少一行、可操作

- **测试策略** 是否与变更风险匹配；改对外 API、路由或鉴权 时 不得 选「不适用」（至少「建议有测」，高敏用「必须自动化」）
- 是否含 **合并前必绿** 条

缺项 → **阻塞**，回到任务单补全后再审。**零阻塞** 也要写记录：写明「已核对哪些项、可进入实现」。

12.5 高敏变更：建议「独立复检」

改 对外 API、流式协议、鉴权 时，除审查签收外，建议 **独立复检**：只读 **diff 摘要**、**自检输出**、**验收表**，逐项 pass/fail（宜新开对话，或同对话换角色说明且只贴三件套，见 §14.5）。精神类似 **审查者与实现者分开**（全自动编排系统如 Ralph Loop 同样强调这一原则）——不必上封闭循环编排器，纪律写在任务单即可。

示例项目规则：**必须自动化** 且动到 API/契约的 task，收尾前须有独立复检书面记录（2026-05 落地）。

12.6 谁来做

场景	建议
个人项目	未来的你（隔日冷读）或同伴
团队	指定 Reviewer

12.7 与 Code Review 的关系

Review 看代码质量；协作流程额外要求：**对照任务单 + CI + 留签收**。卷一 §4 的分工不变；本卷强调 **结论可检索**。

12.8 更深一层：谁负责、如何负责、如何修复

用 AI 辅助研发后，合并变快了，复盘却常变难。许多团队卡在三问上：

难题	核心问题	没有结构时会怎样
谁负责	这行代码、这条决策算谁的	只有聊天；Git 作者是 Agent；说不清谁批准
如何负责	凭什么说「该做的都做了」	说不清验收口径、测没测、有没有越界
如何修复	CI 或线上红了，从哪切入	不知道动过哪些模块、当时为何那样改

这三件事，本质是 **归因 → 尽职 → 修复路径**。单靠模型变强解决不了；需要 **外置、可读、可核对** 的轨迹——笔者称之为 **白盒轨迹**（交付过程与依据应尽量完全可读、可核对，**不依赖模型内部状态**）：模型某一步怎么想仍可能说不清，但 **谁批准、按什么规格、测了什么** 应留在白盒里。

```
flowchart LR
    subgraph 痛点["行业痛点"]
        Q1["谁负责? "]
        Q2["如何负责? "]
        Q3["如何修复? "]
    end

    subgraph 栈["结构地图 + 签收 + 验证"]
        C["冷层: 结构地图<br/> (= 卷二技术图谱) "]
        W["温层: 任务单 + 书面签收"]
        V["合并前 CI / 测试"]
    end

    Q1 --> W
    Q2 --> W
    Q2 --> V
    Q3 --> C
    Q3 --> W
    Q3 --> V

    style W fill:#fff8e1
```

三问	本卷 + 卷二怎么补
谁负责	人 在审查签收上拍板；Agent 是工具，不是责任主体
如何负责	任务单字段齐全 + 任务审核记录 + 合并前必绿
如何修复	卷二结构地图（本卷称冷层）定位模块与影响面；温层 查上次回顾摘要；Git 看 diff；CI 看哪条测试红

12.9 可追责包：交付物不止是 PR

理想情况下，一轮交付除了 PR / 代码 / 测试结果，还应能 一键追到依据：

依据项	是什么
任务单	验收、非范围、失败路径
书面审查	开工前审核 + 合并前签收
结构地图	本轮改动的入口与影响面（卷二技术图谱；本卷分层里称冷层）
版本记录	提交历史与合并记录
机器验收	CI / 单测日志

```
flowchart TD
    TASK["人类描述的任务"] --> COLD["冷层：入口与影响面"]
    COLD --> EXEC["执行：人 + Agent"]
    EXEC --> WARM["温层：审查与回顾摘要"]
    WARM --> DELIVER["交付：PR + 可追责包"]
    DELIVER --> PACK["改了什么 / 为何 / 谁审 / 测了什么"]

    style WARM fill:#fff8e1
```

卷二给结构地图，卷三给签字依据——叠在一起，才谈得上「敢用 AI 规模化改代码」。验收口径在任务单（温层）；结构地图见卷二（本卷分层里称冷层）。热层（运行时事件网）面向长跑与在线系统，团队内部 AI 编程现阶段通常不必做（§14.8）。

💡 常见疑问

- Agent 写的代码，出现事故谁负责谁解决？ → 算 签收合并的人与组织；Agent 是执行工具。所以 书面签收不可省，也不是形式主义。
- 可追责包能消除模型黑盒吗？ → 不能逐 token 解释模型；但能证明 按什么规格、谁批准、测了什么——这在工程与合规上通常才是「负责」的主战场。

13. SDD 阶段流：从需求澄清到合并

13.1 本节要回答什么

卷一 §2.2 阶段骨架 如何变成 可执行流水线。

13.2 阶段图

```
flowchart LR
    Clarify["需求澄清<br/>人"] --> Audit["任务审核<br/>人"]
    Audit --> Impl["实现<br/>人+Agent"]
    Impl --> Self["自检<br/>人+Agent"]
    Self --> CI["合并前 CI<br/>全绿?"]
    CI -->|否| Impl
    CI -->|是| Sign["审查签收<br/>人"]
    Sign --> Merge["合并"]
    Sign -->|可选| Re["冷静复检<br/>人"]
    Re --> Merge
```


13.3 阶段表

阶段	主要谁	产出	图谱 / CI
需求澄清	人 + Agent 辅助	任务单初稿	可选：主图/子图入口
任务审核	人	书面审核结论	核对 §11 字段
实现	人 + Agent	代码、文档	按 §11.5：必须自动化 时先有可失败测例再改实现；建议有测 时同 PR 补测；按入口改图（卷二）
自检	执行者	命令输出 + 验收摘要	本地或 CI 预跑
合并前 CI	机器	workflow 全绿	单测、lint、图谱门禁等
审查签收	人	签收记录	CI 绿 + 任务单勾选
合并	人	进主干	—
（可选）冷静复检	独立视角	隔日复核	高敏建议

13.4 SDD 一句

规格驱动：先写清 验收与边界（任务单 ± SPEC），再让 Agent 改实现——代码是规格的可执行表达。

13.5 机器与人

角色	管什么
CI / 单测	行为不漂移
图谱门禁（若有）	流程图、入口清单与代码 大致一致（卷二 §9.5）
人	敢不敢合

CI 绿了，仍要人看 范围与风险。

13.6 与卷一阶段骨架的对照

卷一文字版：

需求澄清 → 任务审核 → 实现 → 自检 → [CI 全绿?] → (否, 回实现) → 审查签收 → 合并

本卷增补：任务审核 可阻塞实现；独立复检 为可选加强，不替代签收。

13.7 收尾后与「温层」(指针)

任务 归档 / 收尾 时留下的 任务单终稿、审查记录、回顾摘要，在分层愿景里属于 温层——协作轨迹：记的是「这次为何这样改、谁签收」，不必重画整张结构地图（结构地图属冷层，即卷二技术图谱；只在模块连线真的变了时才更新，见卷二）。

```
flowchart TD
    START["按任务单实现 + 测试"] --> REVIEW["书面审查落盘"]
    REVIEW --> CLOSE["任务归档 / 收尾归档"]
    CLOSE --> WARM["温层：跨轮回顾摘要<br/>（远期可挂到地图节点）"]
    WARM --> NEXT["下次改同一模块前<br/>先读到上次结论"]

    style START fill:#fff8e1
    style NEXT fill:#c8e6c9
```

卷四 展开如何把回顾摘要蒸馏成 经验卡片 / 编译摘要，让下一轮 少翻全文；仍 不替代 卷二地图答「改哪里」。热层属远期场景，本卷不展开。

14. 半自动协作：何时链式、何时必须人工闸

14.1 本节要回答什么

Agent 能否 一条对话 里连续写任务、改代码、跑自检？何时 必须停 等人？

14.2 半自动 ≠ 全自动

可以链式	仍须人做
小改动：实现 → 自检 → 整理审核材料	任务审核 终局
工具内切换「角色说明」	审查签收
自动跑测试、贴日志	合并主干

参考 Ralph Loop 一类全自动编排系统：四步封闭循环（规划→实现→审查→结束），机器可连续跑完一轮。

笔者选型是签收型流程：省重复填表，不省人的签收与合并责任。

14.3 人工闸

任务单可用「待批准 / 已批准」表标记关键决策；Agent 在「待批准」时 应停止 进入实现。

闸（示例名）	典型阻塞	谁改「已批准」
初稿任务单	任务审核、实现	负责人（默认人）
审核后	实现	读过审核记录的人（默认人）
复检后（可选）	收尾、合并	Tech Lead
发布前（可选）	合入主干	维护者

默认由人 把「待批准」改为「已批准」——**未显式授权的 Agent 不得代填**。初期建议坚持默认由人改「已批准」：人工闸还没走顺时，不要让实现 Agent 自行批准。

进阶：在 **OpenClaw** 等带 **总调度权限** 的模式里，理论上可以给某一 Agent **书面授权** 代填「已批准」（须可审计：谁授权、何时、依据哪份审核记录）。闸走顺之后，加上有权限的签收代理通常很快；**阶段顺序与任务单字段不变**。笔者尚未在自家流程里完整落地这一层，但整条 SDD 流水线不必因此改写。

§13 阶段图里的「人」，文中均指 **默认执行者**；若使用已授权、留痕的签收代理，仍须满足上段的授权与审计要求。

 常见疑问

- 以后能让总调度 Agent 代点「已批准」吗？→ 可以作为进阶，但须显式授权与留痕；默认仍建议人。流程不变，变的是 **谁被授权点**这一下。

14.4 何时开链式、何时关

类型	建议
小 bug、文档、单文件	可 半自动 ；仍须终轮签收
跨模块、契约、架构	强制人工闸 ；多轮任务审核

半自动省的是 **复制模板**；**不保证** 模型不越界——纪律来自任务单 + 书面记录。

14.5 换上下文（Fresh Context）纪律

任务审核、独立复检时，输入只保留：**任务单、审查记录摘要、diff 要点、自检输出**——**不要** 粘贴整段实现过程长文。交给审核或复检的「三件」：**改了什么、跑了什么命令、验收表结论**。这与「审查者不与实现者共用记忆」同一精神。

新开对话 是最稳的做法；**同一对话里换角色说明** 也可：阶段之间切换「写任务单 / 审核 / 实现 / 自检 / 独立复检」等提示词，到审核或复检环节 **只贴三件套、不贴实现长文**，即在单窗口里做 **逻辑上的 Fresh Context**（与 §14.7 默认路径一致）。

极简示例：粘贴 任务单全文 + `git diff --stat`（或 diff 摘要）+ 自检命令输出 三件套即可开始审核或复检。

14.6 与 Cursor / Claude Code 的关系

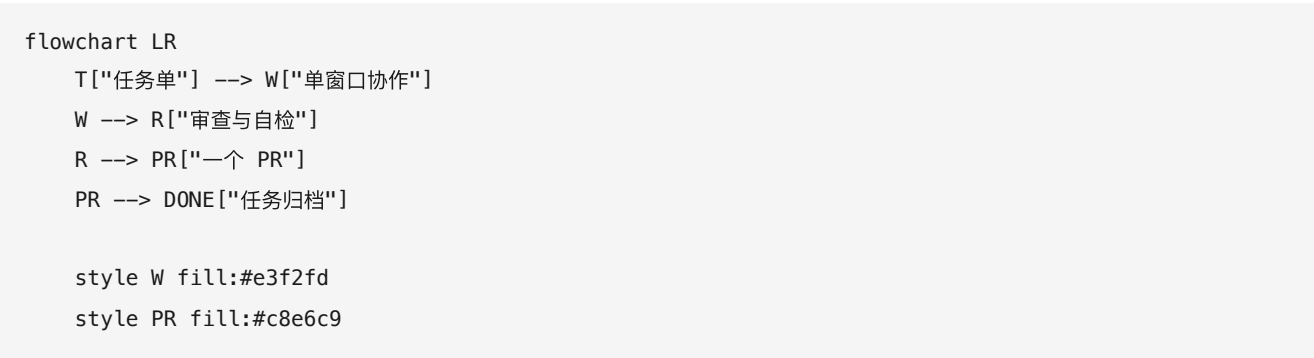
工具可连续对话；流程 来自任务单与签收习惯。换工具时，意图 / 成果 / 验收 字段应可迁移（卷一 §0）。

14.7 默认纪律：一个任务、一个对话、一个 PR

在 任务单 + 地图 + 书面签收 架构下，笔者推荐的 默认路径 是：

一个进行中的任务 → 一个 Agent 对话窗口 → 一条审查与收尾链 → 一个 PR

同一对话可串完整链：从任务单 → 任务审核 → 实现 → 自检 → 审查签收 →（高敏可选）独立复检 → 归档。不必 为每个阶段另开 Agent 或另开窗口——阶段之间 换帽 / 换角色说明 即可（§14.5）。独立复检可在同窗口完成，前提是 只审 diff/日志/验收表，不共用实现过程记忆；高敏或上下文对不齐时，再另开对话。



问题	建议
同一任务开多个对话分工？	避免——上下文分裂，收尾与可追责包会对不齐
两个无关任务同时要交？	可以两个对话，但须 独立 Git 分支与工作目录，勿共用同一份 checkout
多 Agent 自动派活长跑？	远期场景；与本节「一任务一 PR」不矛盾——那是编排器拆成 多个任务

这与「用 AI 盯同事谁改了哪个文件」无关：谁改了哪行 看 Git；是否符合任务范围 看 任务单与审查；改 A 影响谁 看 卷二地图。

14.8 现阶段：冷 + 温已够支撑研发

若场景是 团队内部用 AI 协作写代码、尚无大量真实用户流量，卷二结构地图（本卷称冷层）+ 本卷签收与任务单（温层）+ Git/CI 通常已够，不必 为协作单独上「运行时事件记忆」：

```
flowchart LR
    subgraph 现在够用
        C["冷层：结构地图<br/>（卷二已讲）"]
        W["温层：任务单 + 书面签收"]
        G["Git + CI"]
    end

    subgraph 不必现在做
        H["热：运行时事件网"]
    end

    H -.->|"收尾压缩（远期）"| W
    C --> W
    G --> W

    style H fill:#fce4ec,stroke-dasharray: 5 5
```

15. 存量 / 无 CI：阶段流降级

15.1 本节要回答什么

老项目、还没有 CI 时，如何 不降质地缩水。

15.2 与卷一 §5 衔接

先补 手动门禁（命令写进任务单），再逐步上 CI。

15.3 降级对照

有 CI	无 CI 降级
PR 上 pytest / lint 全绿	合并前 本地 跑同一命令，要点贴进 签收记录
图谱 check workflow	合并前 手动 export/check（命令写进任务单）
任务审核书面记录	不可省；模板可缩短
独立复检	一人团队：合并后 issue / 日记自检

15.4 与产品工单叠加

任务单 = 工程交付 依据；工单 = 产品沟通。先 一轮 手动闭环，再补 CI。

15.5 领域结构检查（进阶）

除通用单测与图谱契约门外，可为 **高频错误响应**、**流式事件名** 等增加 **小脚本结构检查**，与 `pytest` 并列进 CI——思路类似「报告结构 Linter」：专查最容易出现结构偏差的 JSON/API 形状，域按你业务自定。

笔者在 **示例后端 API 项目** 中已落地首条：**结构化错误响应** 的必填字段由注册表 + 脚本校验（理论对齐 P1, PR #92），收尾复检 PR #93。你的仓库可从「一种最容易出现结构偏差的 JSON 形状」起步，不必一次做全。

16. 结语

```
flowchart TB
    subgraph v2["卷二 · 地图"]
        M["图谱入口 · 子图 · CI 不漂移"]
    end
    subgraph v3["卷三 · 交接单"]
        T["任务单字段"]
        W["书面审核 · 签收"]
        C["合并前 CI"]
    end
    D["可闭环交付"]
    M --> D
    T --> D
    W --> D
    C --> D
```

卷二给 **结构地图**（本卷分层里称 **冷层**）；卷三给 **签收纪律**（**温层**）。叠在一起，才适合 **Auto 模型**、**预算有限** 的日常里 **稳定合并**。

AI 编程要规模化，关键不只是让模型多写几行代码，而是 **每次交付都能回答：谁签的、依据什么、影响多大、怎么回滚** (§12.8)。先建好 **白盒轨迹的地基**，再谈把更多执行交给 AI——顺序反了，只会更快产出 **不敢合并、也不敢背锅** 的改动。

下一卷 **卷四**：展示 **一整轮专题从 SPEC 到归档** 的完整例子，以及收尾后 **温层摘要如何压缩成经验卡片**（仍不替代图谱与任务单）。冷/温/热完整愿景见独立篇（可选读；发表版链接待补）。